Compter, filtrer, trier et mesurer

Les outils qu'un ML Engineer utilise pour explorer des données : parcours (`enumerate`, `zip`), compréhensions, tri, comptage (`Counter`, `defaultdict`) et coût d'un algorithme (Big O). Chaque outil est relié au code d'ExamPredict AI.

1. Parcourir : enumerate et zip
2. Les compréhensions
3. Trier : sorted et key
4. Compter : dict, defaultdict, Counter
5. Les 4 gestes : chercher, compter, filtrer, classer
6. Complexité : Big O
7. Lecture du code d'ExamPredict
8. Exercices corrigés
9. Mission : 10 000 résultats
10. À savoir par cœur

> **Pourquoi cette fiche**
>
> Avant d'entraîner un modèle, un ML Engineer passe des heures à **explorer les données** : combien d'exemples par catégorie ? Quelles valeurs reviennent le plus ? Y a-t-il des doublons ? On appelle cette étape l'**EDA** (*Exploratory Data Analysis*). Tous les outils de cette fiche servent à ça. Plus tard, la bibliothèque `pandas` fera la même chose en une ligne, mais il faut d'abord comprendre ce qu'elle fait en coulisses.

1. Parcourir : `enumerate` et `zip`

Rappel : la boucle `for`

```
notes = [12, 15, 9]
for n in notes:        # n prend successivement 12, puis 15, puis 9
    print(n)
```

En clair : « pour chaque élément de la liste, fais ceci ». Le nom `n` est une étiquette qui se déplace d'élément en élément.

> **Définition 1.1** *(enumerate)*
>
> `enumerate(seq, start=0)` produit des couples `(position, élément)`.

```
themes = ["forces", "énergie", "ondes"]

# ✗ Style débutant
for i in range(len(themes)):
    print(i, themes[i])

# ✓ Style pro
for i, theme in enumerate(themes, start=1):
    print(i, theme)        # 1 forces / 2 énergie / 3 ondes
```

En clair : `enumerate` te donne en même temps le **numéro** et l'**élément**. C'est parfait pour afficher un classement « 1., 2., 3. ».

> **Définition 1.2** *(zip)*
>
> `zip(a, b)` parcourt deux séquences **en parallèle** et produit des couples `(a[0], b[0])`, `(a[1], b[1])` … Il s'arrête à la fin de la plus courte.

```
eleves = ["Ana", "Bilal", "Chloé"]
notes  = [14, 9, 17]
for nom, note in zip(eleves, notes):
    print(nom, note)              # Ana 14 / Bilal 9 / Chloé 17

bulletin = dict(zip(eleves, notes))  # {'Ana': 14, 'Bilal': 9, 'Chloé': 17}
```

En clair : `zip` fonctionne comme une fermeture éclair. Il accroche le 1er avec le 1er, le 2e avec le 2e, et ainsi de suite.

> **Dans le métier**
>
> Pour comparer les **vraies réponses** aux **prédictions** d'un modèle : `for vrai, predit in zip(y_true, y_pred):`. C'est la base du calcul de la précision.

2. Les compréhensions

Définition 2.1 *(compréhension de liste)*

`[expression for x in iterable if condition]` construit une **nouvelle liste** en une ligne. La partie `if` est facultative.

```
notes = [12, 7, 15, 9, 18]

# Version longue
reussis = []
for n in notes:
    if n >= 10:
        reussis.append(n)

# Compréhension : exactement la même chose
reussis = [n for n in notes if n >= 10]     # [12, 15, 18]
sur_1   = [n / 20 for n in notes]           # [0.6, 0.35, 0.75, 0.45, 0.9]
```

En clair : on lit la compréhension comme une phrase : « donne-moi **n**, pour chaque **n** dans **notes**, **si** n ≥ 10 ».

Définition 2.2 *(compréhensions de dict et de set)*

```
{cle: valeur for x in iterable}      # dictionnaire
{expression for x in iterable}       # ensemble (sans doublons)
```

```
bulletin = {"Ana": 14, "Bilal": 9, "Chloé": 17}
admis    = {nom: n for nom, n in bulletin.items() if n >= 10}   # {'Ana': 14, '(
themes   = {t.strip().lower() for t in [" Forces", "forces ", "Ondes"]}  # {'for
```

Lien avec la fiche F1

Une compréhension crée **toujours un nouvel objet**. Elle ne modifie jamais la liste d'origine. C'est donc la façon la plus propre de corriger l'effet de bord de `normaliser_notes` : `return [n / 20 for n in notes]`.

Piège 2.3 *(lisibilité)*

Si une compréhension ne tient pas sur une ligne ou contient plusieurs `if` / `for`, on revient à une boucle normale. Le code se lit plus souvent qu'il ne s'écrit.

3. Trier : `sorted` et `key`

	`SORTED(SEQ)`	`LISTE.SORT()`
Renvoie	une **nouvelle** liste triée	`None`
Original	intact	**modifié** sur place (mutation)
Accepte	tout itérable (liste, tuple, dict…)	uniquement les listes

```
notes = [12, 7, 15]
sorted(notes)                 # [7, 12, 15]
sorted(notes, reverse=True)   # [15, 12, 7]   du plus grand au plus petit
notes = notes.sort()          # X ERREUR CLASSIQUE : notes vaut maintenant None
```

Définition 3.1 *(clé de tri)*

`sorted(seq, key=f)` trie les éléments selon la valeur de `f(élément)`, et non selon l'élément lui-même.

Définition 3.2 *(lambda)*

`lambda x: expression` est une **mini-fonction sans nom**, écrite sur une ligne. `lambda p: p[1]` est équivalent à :

```
def f(p):
    return p[1]
```

En clair : on l'utilise quand on a besoin d'une petite fonction à un seul endroit, souvent pour dire à `sorted` **sur quoi** trier.

```
bulletin = {"Ana": 14, "Bilal": 9, "Chloé": 17}

# .items() donne des couples (nom, note) : ('Ana', 14), ('Bilal', 9)…
classement = sorted(bulletin.items(), key=lambda p: p[1], reverse=True)
# [('Chloé', 17), ('Ana', 14), ('Bilal', 9)]

mots = ["ondes", "forces", "énergie"]
sorted(mots, key=len)           # ['ondes', 'forces', 'énergie']  trié par longu
```

---

4. Compter : `dict`, `defaultdict`, `Counter`

Problème : on a une liste de thèmes et on veut savoir **combien de fois chacun apparaît**. Trois méthodes, de la plus manuelle à la plus pro.

```
themes = ["forces", "ondes", "forces", "énergie", "forces", "ondes"]
```

Méthode 1 : dictionnaire à la main

```
compte = {}
for t in themes:
    if t in compte:
        compte[t] += 1
    else:
        compte[t] = 1
# {'forces': 3, 'ondes': 2, 'énergie': 1}
```

Variante plus courte : `compte[t] = compte.get(t, 0) + 1`. `.get(t, 0)` veut dire « la valeur de t, ou 0 si t n'existe pas encore ».

> **Piège 4.1** *(KeyError)*
>
> Écrire `compte[t] += 1` sans vérification provoque `KeyError` la première fois qu'on rencontre `t` : on ne peut pas ajouter 1 à une case qui n'existe pas.

Méthode 2 : `defaultdict`

Définition 4.2

`defaultdict(int)` est un dictionnaire qui crée automatiquement une case valant `int()`, c'est-à-dire `0`, dès qu'on demande une clé absente. `defaultdict(list)` crée une liste vide `[]`.

```
from collections import defaultdict

compte = defaultdict(int)
for t in themes:
    compte[t] += 1           # plus de KeyError possible

# Regrouper : quels élèves par note ?
par_note = defaultdict(list)
for nom, n in [("Ana", 14), ("Bilal", 9), ("Chloé", 14)]:
    par_note[n].append(nom)  # {14: ['Ana', 'Chloé'], 9: ['Bilal']}
```

Méthode 3 : `Counter`

Définition 4.3

`Counter(iterable)` compte chaque élément en une ligne. `.most_common(k)` renvoie les `k` éléments les plus fréquents, triés par fréquence décroissante.

```
from collections import Counter

compte = Counter(themes)        # Counter({'forces': 3, 'ondes': 2, 'énergie': 1
compte["forces"]                # 3
compte["chimie"]                # 0   (pas de KeyError)
compte.most_common(2)           # [('forces', 3), ('ondes', 2)]
```

BESOIN	OUTIL
Compter des éléments	`Counter`
Regrouper des éléments par catégorie	`defaultdict(list)`
Accumuler une somme par catégorie	`defaultdict(int)` ou `defaultdict(float)`
Comprendre ce qui se passe (examen, entretien)	le dict à la main

Dans le métier

`Counter(labels)` est souvent la toute première ligne d'une EDA : on vérifie si les classes sont **équilibrées**. Si 98 % des transactions sont « normales » et 2 % « fraude », un modèle qui répond toujours « normal » obtient 98 % de précision… tout en étant inutile.

5. Les 4 gestes : chercher, compter, filtrer, classer

GESTE	QUESTION	CODE TYPE
Chercher	Est-ce présent ? Où ?	`x in seq` · `seq.index(x)` · `max(seq)` · `min(seq)`
Compter	Combien ?	`len(seq)` · `seq.count(x)` · `Counter(seq)` · `sum(1 for x in seq if …)`
Filtrer	Lesquels respectent la règle ?	`[x for x in seq if condition]`
Classer	Dans quel ordre ?	`sorted(seq, key=…, reverse=True)` · `Counter.most_common(k)`

```
notes = [12, 7, 15, 9, 18, 15]
max(notes)                                  # 18
sum(notes) / len(notes)                     # 12.67   moyenne
sum(1 for n in notes if n >= 10)            # 4       nombre de réussites
max(bulletin, key=bulletin.get)             # 'Chloé' clé ayant la plus grande valeu
```

6. Complexité : Big O

Définition 6.1 *(complexité)*

La **complexité** d'un algorithme décrit comment son nombre d'opérations grandit quand la taille `n` des données grandit. La notation **O(…)** (« grand O ») garde seulement le terme dominant.

En clair : Big O ne mesure pas des secondes. Il répond à la question : « si j'ai **10 fois plus** de données, mon programme sera-t-il 10 fois plus lent ? 100 fois ? pas plus lent du tout ? »

NOTATION	NOM	IMAGE	EXEMPLE PYTHON	OPÉRATIONS POUR N = 10 000
O(1)	constant	ouvrir le tiroir étiqueté	`d[k]`, `x in set`, `liste[i]`, `append`	1
O(n)	linéaire	lire toutes les pages une fois	une boucle `for`, `x in liste`, `sum`, `max`, `Counter`	10 000
O(n log n)	quasi linéaire	trier un jeu de cartes intelligemment	`sorted`, `.sort()`	≈ 133 000
O(n²)	quadratique	chacun serre la main de tout le monde	deux boucles imbriquées sur les mêmes données, `liste.count` dans une boucle	100 000 000

> **Proposition 6.2** *(liste ou set pour chercher)*
>
> `x in liste` est en **O(n)** : Python compare x à chaque élément, un par un. `x in ensemble` (ou `x in dict`) est en **O(1)** en moyenne : Python calcule directement l'emplacement de x (c'est le principe du *hachage*).

```
# ✗ O(n²) : pour CHAQUE élément, .count() relit TOUTE la liste
compte = {}
for t in themes:
    compte[t] = themes.count(t)

# ✓ O(n) : une seule lecture de la liste
compte = Counter(themes)
```

Méthode 6.3 *(mesurer un temps d'exécution)*

```
import time

debut = time.perf_counter()          # chronomètre précis, en secondes
# ... le code à mesurer ...
duree = time.perf_counter() - debut
print(f"{duree:.4f} s")              # 4 chiffres après la virgule
```

Dans le métier

Un dataset ML compte facilement 1 million de lignes. Un code $O(n^2)$ qui prend 0,1 s sur 1 000 lignes prendra **plus d'une journée** sur 1 million. Le même calcul en $O(n)$ prend moins d'une seconde. Reconnaître une boucle imbriquée cachée, c'est une compétence d'entretien d'embauche.

---

7. Lecture du code d'ExamPredict

Ton projet utilise déjà tous ces outils. Voici des extraits de `src/analyzer.py` et `src/main.py` :

```
# analyzer.py — find_concepts
found = Counter()
...
found[concept] += occurrences          # Counter : pas de KeyError, part de 0

# analyzer.py — analyze_documents
document_counts = defaultdict(int)     # dans combien de documents chaque thème
document_counts[concept] += 1

# analyzer.py — calculate_priorities
return sorted(results, key=lambda x: x["score"], reverse=True)   # classement pa

# main.py — display_results
for position, result in enumerate(results[:20], start=1):        # top 20 numéro
```

À examiner à 14 h

`find_concepts` appelle `text.count(keyword)` pour chaque mot-clé. Demande-toi : `"force"` est-il aussi compté dans `"forces"` ? `"ph"` est-il compté dans `"physique"` ? Et dans `predictor.py`, que fait `max(...)` si le dictionnaire est vide ?

---

8. Exercices corrigés

Méthode : écrire la réponse dans un fichier `.py`, l'exécuter, puis seulement comparer avec le corrigé.

Exercice 1 ★ enumerate

Afficher `1. forces`, `2. énergie`, `3. ondes` à partir de `themes = ["forces", "énergie", "ondes"]`.

```
for i, t in enumerate(themes, start=1):
    print(f"{i}. {t}")
```

Exercice 2 ★ compréhension avec filtre

À partir de `notes = [12, 7, 15, 9, 18]`, construire la liste des notes inférieures à 10, puis la liste des notes sur 100.

```
echecs = [n for n in notes if n < 10]      # [7, 9]
sur_100 = [n * 5 for n in notes]           # [60, 35, 75, 45, 90]
```

Exercice 3 ★ zip

Avec `y_true = [1, 0, 1, 1]` et `y_pred = [1, 0, 0, 1]`, calculer la proportion de bonnes prédictions (la *accuracy*).

```
bons = sum(1 for v, p in zip(y_true, y_pred) if v == p)   # 3
accuracy = bons / len(y_true)                              # 0.75
```

Exercice 4

★★ trier un dictionnaire

Classer `{"forces": 12, "ondes": 30, "chimie": 7}` du thème le plus fréquent au moins fréquent.

```
sorted(d.items(), key=lambda p: p[1], reverse=True)
# [('ondes', 30), ('forces', 12), ('chimie', 7)]
```

Pourquoi `p[1]` ? Chaque élément de `d.items()` est un couple `(thème, nombre)` : `p[0]` est le thème, `p[1]` est le nombre.

Exercice 5

★★ compter de trois façons

Compter `["A", "B", "A", "C", "A", "B"]` avec un dict à la main, avec `defaultdict`, puis avec `Counter`. Afficher les 2 plus fréquents.

```
c1 = {}
for x in data:
    c1[x] = c1.get(x, 0) + 1

c2 = defaultdict(int)
for x in data:
    c2[x] += 1

c3 = Counter(data)
print(c3.most_common(2))     # [('A', 3), ('B', 2)]
```

Exercice 6

★★ reconnaître la complexité

Donner la complexité de chaque bloc (`data` a n éléments) :

```
# (a) total = sum(data)
# (b) premier = data[0]
# (c) for x in data:
          for y in data:
              if x + y == 10: ...
# (d) top = sorted(data)[:3]
# (e) vus = set(data); 42 in vus
```

Corrigé. (a) O(n). (b) O(1). (c) O(n²), deux boucles imbriquées. (d) O(n log n), c'est le tri qui domine. (e) O(n) pour construire le set, puis O(1) pour chaque recherche.

Exercice 7

★★★ éviter le O(n²)

Ce code trouve les éléments de `a` présents dans `b`. Quelle est sa complexité ? Réécris-le en O(n).

```
communs = [x for x in a if x in b]     # b est une liste
```

Corrigé. `x in b` coûte O(n) et il est fait n fois, donc O(n²). On convertit d'abord `b` en set :

```
b_set = set(b)                          # O(n), une seule fois
communs = [x for x in a if x in b_set]  # O(n) × O(1) = O(n)
```

9. Mission : analyser 10 000 résultats

Énoncé (pas de corrigé : on le corrige ensemble)

Le fichier `exercices/jour1/resultats.txt` contient 10 000 lignes. Chaque ligne est le thème d'une question d'examen. Certaines lignes sont « sales » : majuscules, espaces en trop, lignes vides. Dans `mission_f2.py` :

1. Lire le fichier et obtenir la liste des lignes.
2. **Nettoyer** avec une compréhension : enlever les espaces (`.strip()`), mettre en minuscules (`.lower()`) et retirer les lignes vides.
3. Afficher le nombre de résultats valides et le nombre de thèmes **distincts**.
4. Afficher le **top 5** des thèmes les plus fréquents, numéroté avec `enumerate` , avec leur pourcentage.
5. **Comparer deux méthodes** de comptage avec `time.perf_counter` : la méthode O(n²) avec `.count()` dans une boucle, et la méthode `Counter` . Afficher les deux durées et le rapport entre elles.
6. Écrire en commentaire la complexité de chaque méthode et expliquer l'écart mesuré.

Pour lire le fichier : `with open("resultats.txt", encoding="utf-8") as f: lignes = f.readlines()` . Le `with` referme le fichier automatiquement (vu en détail demain).

10. À savoir par cœur

1 `enumerate(seq, start=1)` donne `(numéro, élément)` . `zip(a, b)` parcourt deux séquences en parallèle.

2 `[expr for x in seq if cond]` construit une **nouvelle** liste. Même logique avec `{k: v for …}` et `{x for …}` .

3 `sorted()` renvoie une nouvelle liste. `.sort()` modifie la liste et renvoie `None` .

4 `key=lambda p: p[1]` trie sur le 2e élément de chaque couple. `reverse=True` trie du plus grand au plus petit.

5 `Counter` sert à compter, `.most_common(k)` à obtenir le top k, et `defaultdict(list)` à regrouper.

6 `d[k] += 1` sur une clé absente provoque `KeyError` . Les remèdes : `.get(k, 0)` , `defaultdict` ou `Counter` .

7	Les quatre complexités : O(1) < O(n) < O(n log n) < O(n²).
8	`x in liste` est en O(n). `x in set` ou `x in dict` est en O(1).
9	Deux boucles imbriquées, ou `.count()` / `in liste` dans une boucle, donnent du O(n²) : il faut les repérer et les éviter.
10	On mesure un temps d'exécution avec `time.perf_counter()`.
